

04 — Persistere oggetti: la forma su disco non è la forma in memoria

La domanda a cui risponde: perché un oggetto salvato in un database non torna indietro uguale a com'era, e cosa deve fare il codice per rimetterlo in piedi. Il caso di studio è un bug vero di questo progetto.

1. Il bug, per cominciare dal fondo

Aprendo un mazzo salvato e premendo  su una carta, l'app rispondeva:

Sostituzione non riuscita: undefined is not an object (evaluating 'carta.idSet')

Il messaggio dice due cose: che qualcuno ha letto `.idSet` su un valore inesistente, e che quel qualcuno si chiamava `carta`. Il punto è `src/engine/dispensa.js`:

```
export function chiaveCarta(cart) {  
  return `${cart.idSet ?? '?'}:${cart.numero ?? normalizzaNome(cart.nome)}`;  
}
```

Chiamata da `disponibilitaResidua()` in `src/engine/alternative.js`:

```
for (const voce of mazzo.carte) {  
  dispensa.preleva(voce.carta, voce.quantita); // ← voce.carta era undefined  
}
```

Nota bene: `?? '?'` **non protegge da niente qui**. Difende dal caso “carta senza `idSet`”, non dal caso “carta che non c'è”. È un errore di lettura molto comune: si guarda un `??` e si pensa “ok, è gestito”.

2. Perché `voce.carta` non c'era

Il salvataggio (`src/data/mazzi-salvati.js`) scriveva ogni carta **appiattita**:

```
carte: m.carte.map((c) => ({ quantita: c.quantita, nome: c.carta.nome, idSet: ... })))
```

mentre tutto il resto del programma lavora sulla forma **annidata** `{carta: {...}, quantita}`. Finché il mazzo restava quello appena generato in memoria, nessuno se ne accorgeva: il bug si presentava **solo** dopo un giro sul disco. È la firma tipica di questa classe di errori — funziona finché non salvi e ricarichi.

Chi viene da Java riconosce lo schema: è la differenza fra **entità** e **DTO**. Il DTO è piatto perché deve viaggiare; l'entità è ricca perché deve lavorare. Il mestiere di tradurre l'uno nell'altro ha un nome (*mapper*) e in Java te lo ricorda il compilatore: un `CartaDto` non è assegnabile a una `Carta`. In JavaScript i due oggetti sono *lo stesso tipo* — nessuno — e l'errore emerge solo quando qualcuno legge un campo che non c'è, magari tre moduli più in là.

3. La cura: una sola coppia andata/ritorno

La correzione non è mettere una difesa dove è esploso (`voce.carta?.idSet`): quella nasconderebbe il problema lasciando il mazzo mezzo rotto. La cura è avere **due funzioni pure e simmetriche**, in un punto solo:

```
export function istantanea(piano, opzioni, nome, creatoIl) { /* → forma piatta */ }  
export function idrataPiano(record) { /* → forma annidata */ }
```

Tre proprietà valgono più della loro implementazione:

1. **Simmetria.** Se `istantanea()` smette di salvare un campo, `idrataPiano()` non può inventarlo: le due funzioni si leggono una accanto all'altra e l'omissione salta all'occhio. Per questo la lista `CAMPI_CARTA` è una costante e non un elenco sparso.
2. **Idempotenza.** `idrataPiano(idrataPiano(x))` deve dare `x`. Serve davvero: la funzione si applica a piani che a volte sono già idratati, e senza questa proprietà bisognerebbe sapere *da dove viene* ogni piano prima di toccarlo.

3. **Purezza.** Nessun IndexedDB dentro: sono funzioni che prendono un oggetto e ne restituiscono un altro, quindi si provano in Node senza browser (`tests/mazzi-salvati.test.js`). Il confine è lo stesso che CLAUDE.md impone a `src/engine/`.

Il dettaglio che morde: `null` non è `undefined`

IndexedDB conserva i `null` fedelmente. Ma il motore scrive `carta.idSet ?? '?'`, e `null ?? '?'` restituisce... `'?'`? No: ?? scatta **sia** su `null` **sia** su `undefined`, quindi qui andrebbe bene. Il guaio è altrove:

```
`${carta.idSet}:${carta.numero}` // idSet null → "null:12"
```

Una chiave `"null:12"` non corrisponde a nessuna carta in dispensa, e il risultato non è un errore ma un **silenzio**: zero copie disponibili, nessuna proposta di sostituzione, nessun messaggio. Per questo `senzaNulli()` elimina i campi vuoti invece di conservarli: *assente* e *presente ma nullo* devono restare la stessa cosa in tutte e due le direzioni.

In Java il tema è identico (`Optional`, o le colonne `NULL` di JDBC), ma lì il tipo ti obbliga a decidere. Qui la decisione va presa a mano, una volta, nel punto di frontiera.

Cosa NON si salva

Il record su disco non contiene analisi né `indiceEvoluzioni`: sono fotografie della **collezione**, non del mazzo, e pesano centinaia di kB uguali per ogni salvataggio. Vengono ricostruiti da `apriSalvato()` in `src/app/vista-mazzi.js` con i dati di oggi — che è anche la semantica giusta: una sostituzione pesca dalla collezione di adesso, non da quella di sei mesi fa.

Regola generale: **si persiste ciò che non è ricalcolabile**. Tutto il resto è cache, e una cache dentro un salvataggio permanente è un modo elegante di conservare dati sbagliati.

4. Il pezzo di UI: `<elenco-salvati>`

L'elenco dei mazzi salvati era una stringa di HTML dentro `vista-mazzi.js`. Ora è un Web Component (`src/ui/elenco-salvati/`) con un ingresso e due uscite:

```
elenco.piani = await elencoPiani(); // @Input
elenco.addEventListener('piano-aperto', ...); // @Output
elenco.addEventListener('piano-eliminato', ...); // @Output
```

Rispetto ad Angular mancano due cose e se ne guadagna una:

- manca la **change detection**: `set piani` chiama `#disegna()` a mano, perché nessuno lo farà per noi (vedi il documento 03);
- manca l'**incapsulamento del CSS**: il foglio è globale, quindi le classi sono prefissate `salvati-`. Un `.nome` generico avrebbe colpito anche le liste dei mazzi generati;
- si guadagna che gli eventi sono **eventi DOM veri**: `bubbles: true` e chi sta sopra li ascolta, senza `@Output` da ricablare a ogni livello.

Dentro il componente c'è anche una scelta minuta: **un solo ascoltatore** sul contenitore invece di uno per bottone, con `evento.target.closest('[data-azione]')`. La lista si ridisegna a ogni salvataggio, e riattaccare N ascoltatori ogni volta è lavoro sprecato — è la *delegazione degli eventi*, vecchia quanto jQuery e ancora la cosa giusta.

5. Bonus: quando la piattaforma non fa quello che dice il manuale

Il dialogo del nome (`src/app/chiedi-nome.js`) è nato così:

```
<form method="dialog"> ... <button type="submit" value="ok">Salva</button> </form>
```

È l'idioma da manuale: il form chiude il `<dialog>` da solo e l'esito si legge nell'evento `close`. Provandolo in un browser reale è emerso che quel motore chiude il dialogo **senza emettere né** `submit` **né** `close` — e la Promise del salvataggio restava appesa per sempre: nessun errore, nessun mazzo salvato.

La versione finale rinuncia all'automatismo e ascolta eventi che ci sono ovunque: `click` sul bottone, `keydown` su `Invio`, `cancel` per `Esc`, con una sola via d'uscita (`chiudi()`) protetta da un flag `risolta` perché la `Promise` si risolva una volta sola. Lo stesso inciampo era già annotato in `src/app/sostituzione.js` ("alcuni ambienti non emettono `close`"): due volte non è sfortuna, è un'API su cui non si costruisce.

Morale trasferibile: quando una `Promise` avvolge un evento del DOM, chiediti sempre **chi la risolve se quell'evento non arriva**. Un `reject` mancato non si vede: si vede solo un'app che non risponde più.

6. Esercizi

1. **Rompi la simmetria.** Togli 'attacchi' da `CAMPI_CARTA` e lancia `node --test tests/mazzi-salvati.test.js`. Poi spiega perché il punteggio di forza di un mazzo riaperto cambierebbe, guardando `bilancia.js:94`.
2. **Il null silenzioso.** In `idrataPiano()`, sostituisci `senzaNulli(campi)` con `{...campi}` e scrivi un test che dimostri il danno: partendo da una carta salvata con `numero: null` (capita alle Energie stampate), verifica che chiave produce `chiaveCarta()` e quante copie risultano disponibili in `Dispensa`.
3. **Un secondo uso del componente.** `<elenco-salvati>` è riusabile: mostralo anche nella vista `Regole`, senza duplicare codice, e di' quale singola riga di `vista-mazzi.js` va spostata perché i due elenchi restino allineati dopo un salvataggio.
4. **Domanda secca.** Perché `idrataPiano()` deve essere idempotente, mentre a `istantanea()` non chiediamo la stessa proprietà?